Choosing a Fast Stand-In for a Slow Simulation: a Selection Tool and a 44-Problem Study

[Author names to be supplied by the authors][a]

[a] *[Affiliations and corresponding-author e-mail to be supplied by the authors]*

Abstract

STUB

Keywords: surrogate model, Gaussian process, neural network, reduced-order model, parametric PDE, model selection, learning curve

1. Introduction

Many engineering decisions rest on a simulation that is too slow to run as often as one would like. Choosing the shape of a part, finding how uncertain an input leaves the answer, or fitting a model to measurements can each need thousands of runs. A common remedy is a *surrogate*, or stand-in model: the simulation is run a limited number of times, say a few hundred, a cheap model is trained on the inputs and outputs, and the cheap model then replaces the simulation in all further work. (As an illustration, not a result of this paper: if one run takes an hour, five hundred runs take three weeks on one machine, whereas a trained stand-in answers in milliseconds.)

The practical question is which kind of stand-in to build, and how many runs are enough. One long-standing route compresses the output fields with proper orthogonal decomposition (POD) and regresses the weights of the leading modes on the inputs, with a neural network [1, 2] or a Gaussian process [3]. The other route trains a neural network to produce the whole field [4, 5]. Published comparisons are mixed. A survey of machine-learning solvers for fluid problems found that most papers claiming to beat a standard method used a weak baseline [6]; kernel and Gaussian-process methods have been reported to be competitive with DeepONet [7] and the Fourier neural operator [8] on benchmark problems [9]; and no universal winner was found between neural and polynomial surrogates [10]. A practitioner holding a few hundred runs of their own simulation still has no procedure that tells them

which stand-in to use on *their* data, how far it can be trusted, or how many more runs it would take to reach a target accuracy.

Gap.. Existing comparisons rank methods on fixed problem sets and do not give the practitioner a validated procedure to select a stand-in, to see whether the POD basis limits accuracy, and to forecast the number of runs needed.

Motivation and contributions

1. **A selection tool, SurroSelect** (section 4). It fits a pool of stand-in models, chooses by cross-validation while reporting which models are tied with the best, checks whether the POD basis is the bottleneck, and forecasts the error for more runs from a learning curve. It is free software with a command-line interface (**??**).
2. **A pre-registered evaluation on independent problems** (**??**). Seven criteria were fixed before any model was fitted to eight problems solved by the third-party finite-element library `scikit-fem`; we report which were met and which were not, against a neural network tuned with twice the budget of any other method.
3. **A 44-problem study of where each family wins and why** (**??**). Controls separate the effects of the reduced basis, of the regressor, and of how widely its hyper-parameters are searched; the outcome of a comparison is shown to depend on training budget, tuning effort and basis rank.
4. **An honest record of what did not work** (**??**), including a pilot-data diagnostic that was not confirmed, a control that had to be widened after review, and every deviation from the plan (Supplement, section S4supplement.pdf).

The paper proposes no new regression model; the pool consists of classical methods, and the contribution is the procedure, its validation, and the evidence about when each member wins.

Organisation

Section 2 reviews related work; section 3 fixes notation and assumptions; section 4 describes the method; section 5 the experimental framework; **??** the results; **??** the discussion and limitations; **??** concludes.

2. Related Work

2.1. Reduced-basis surrogates with learned coefficients

Non-intrusive reduced-order models regress the weights of POD modes on the parameters. Hesthaven and Ubbiali used neural networks for this [1], Bhattacharya et al. analysed PCA-based neural operators [2], and Guo and Hesthaven used Gaussian-process regression for nonlinear structural analysis [3]. Bērziņš et al. compared radial-basis-function, Gaussian-process and neural-network regression inside one POD framework for extruder flows and reported several advantages of the Gaussian process [11]; we read only the abstract, and it is the closest precedent for our finding on smooth problems. Mora et al. proposed a hybrid Gaussian-process/neural operator framework [12].

2.2. Neural networks, operators and nonlinear manifolds

DeepONet [7], the Fourier neural operator [8] and the neural-operator framework [13] learn maps between function spaces; autoencoders learn nonlinear latent manifolds for reduced models [**?** 4, 5]. Our parameters are finite-dimensional, so we study direct and bottleneck networks rather than operator-input architectures. Whether a small linear basis can work depends on the decay of the Kolmogorov n-width [14], which is slow for wave problems with discontinuous data [15].

2.3. Kernel and Gaussian-process methods

Gaussian processes are the standard low-dimensional regressor [16]. Batlle et al. develop a kernel framework with a priori error analysis and report it competitive with DeepONet and the Fourier neural operator [9]. A position paper argues that many neural-surrogate successes sit on low-dimensional manifolds where any competent reduced model interpolates well [17].

2.4. Fair comparison, tuning and reporting

McGreivy and Hakim found that 79% of 76 reviewed articles compared with weak baselines [6]. De Hoop et al. study the cost–accuracy trade-off of neural architectures [18]; Westermann et al. compare neural and polynomial surrogates by Pareto frontiers and find no universal winner [10]; a Common Task Framework has been proposed to evaluate scientific machine-learning methods against curated baselines [19]. Random search is an efficient alternative to grid search [20]; adaptive early-stopping schemes such as Hyperband [21] and software such as Optuna [22] reduce tuning cost; reporting results as a function of tuning budget and accounting for run-to-run variance have been advocated [23, 24].

2.5. Surrogate selection, learning curves and software

Williams and Cremaschi compared eight surrogate forms on generated data to look for rules for choosing a form [25]. Learning curves, the error as a function of training-set size, have been surveyed for decision making in supervised learning [26], and error scaling laws have been fitted across data scales in deep learning [27]. General-purpose libraries for model-order reduction [28] and uncertainty quantification [29] exist; we did not compare their features with ours, and the finite-element library `scikit-fem` [30] is used here only to generate independent test problems. PDEBench [31] provides large simulated benchmarks for scientific machine learning; we did not use it (**??**).

2.6. Comparison with prior work

?? lists what each closest work contains, judged from the parts we read (mostly abstracts; Westermann et al. in large part).

Work	Compares kernel/GP/
Hesthaven & Ubbiali [1]	–
Guo & Hesthaven [3]	–
Bērziņš et al. [11]	✓
Batlle et al. [9]	✓
Westermann et al. [10]	✓
McGreivy & Hakim [6]	–
Williams & Cremaschi [25]	–
This paper	✓

2.7. Motivations

(i) Method comparisons are reported on fixed problem sets, so a practitioner cannot tell whether a published ranking applies to their problem. (ii) The outcome of a comparison depends on tuning effort [6, 23], so a tool must make the tuning effort explicit and equal. (iii) Small differences are often ties, so a tool should report ties instead of a single winner. (iv) The accuracy of any POD-based model is bounded by the basis (proposition 2), which should be checked before tuning a regressor. (v) The practical question is the number of runs needed, which learning-curve extrapolation can answer only roughly and must be validated, not assumed.

3. Preliminary Concepts

Problem.. A parametric simulation maps an input vector $\boldsymbol{\mu} \in [0,1]^p$ (rescaled to the unit box) to an output field $\mathbf{u}(\boldsymbol{\mu}) \in \mathbb{R}^m$ sampled on a fixed grid. We are given N runs $\{(\boldsymbol{\mu}_i, \mathbf{u}_i)\}_{i=1}^N$ and want a model $\hat{\mathbf{u}}$ that is accurate on new inputs. Accuracy is the mean relative L_2 error over n held-out inputs,

$$\mathrm{err}(\hat{\mathbf{u}}) = \frac{1}{n} \sum_{j=1}^{n} \frac{\|\hat{\mathbf{u}}(\boldsymbol{\mu}_j) - \mathbf{u}(\boldsymbol{\mu}_j)\|_2}{\|\mathbf{u}(\boldsymbol{\mu}_j)\|_2}. \tag{1}$$

POD basis.. With $\bar{\mathbf{u}}$ the mean of the training fields and $V_r \in \mathbb{R}^{m \times r}$ the r leading left singular vectors of the centred snapshot matrix, a POD model has the form

$$\hat{\mathbf{u}}(\boldsymbol{\mu}) = \bar{\mathbf{u}} + V_r \, \hat{\mathbf{a}}(\boldsymbol{\mu}), \tag{2}$$

where $\hat{\mathbf{a}}$ regresses the coefficients $\mathbf{a} = V_r^\top (\mathbf{u} - \bar{\mathbf{u}})$ on $\boldsymbol{\mu}$.

Regressors.. Gaussian process (GP): the posterior mean with a Matérn-5/2 or squared-exponential kernel and one length-scale per input (ARD), hyper-parameters fitted by maximising the marginal likelihood [16]. *Kernel ridge regression (KRR):* $\hat{a}(\boldsymbol{\mu}) = \sum_i \alpha_i k(\boldsymbol{\mu}, \boldsymbol{\mu}_i)$ with $\boldsymbol{\alpha} = (K + \lambda N I)^{-1}\mathbf{a}$. *Polynomial:* ridge regression on total-degree monomials.

Ridge hat matrix and leave-one-out.. For a ridge-type smoother the fitted values are $H\mathbf{a}$ with $H = K(K + \lambda N I)^{-1}$, and the leave-one-out residual of run i is $(a_i - \hat{a}_i)/(1 - H_{ii})$ (proposition 1).

Tie band.. Two errors are *tied* if their ratio lies within $10^{\pm 0.1} \approx 1.26$; the outcome of a comparison of groups A and B is $\bar{y} = \log_{10}(\mathrm{err}_B/\mathrm{err}_A)$, positive when A is better, and a cell is a tie if $|\bar{y}| \le 0.1$.

Assumptions..

A1 Inputs of training and test runs are drawn independently from the same distribution on the unit box; extrapolation tests (outside the box) are reported separately.

A2 The output grid is the same for every run and the solver is deterministic (no noise).

A3 Runs are exchangeable, so that k-fold cross-validation estimates the error on new runs. This fails for adaptive or clustered designs; the tool does not detect that.

A4 The number of inputs is moderate (up to a few tens in our tests; only five of our 44 problems have ten or more).

PLACEHOLDER fig_framework (generated after Stage 6)

Figure 1: SurroSelect pipeline. The N simulation runs enter a pool of stand-in models; k-fold cross-validation selects the best model and reports which others are tied with it; the basis check compares the model error with the error of projecting held-out fields onto the POD basis; the learning curve forecasts the error for more runs. The example fields are taken from the first test sample of problem `heatcomp_L1` (seed 0, $N = 256$): true field, selected model, and absolute error, as chosen by the rule in **??**.

4. Proposed Methodology: SurroSelect

4.1. Overview

SurroSelect takes the N runs and returns (a) the selected stand-in, (b) the set of models tied with it, (c) a basis check, and (d) a learning-curve forecast (fig. 1, algorithm 1). Every step uses only the training runs.

4.2. Candidate pool

The default pool has seven members, all classical: POD+GP with a Matérn-5/2 kernel and rank 32, with a squared-exponential kernel and rank 32, and with Matérn-5/2 and rank 8 (GP on the four leading modes with marginal-likelihood fitting, remaining modes sharing the fourth kernel); POD+KRR-wide, whose kernel, length-scale (0.05–$15\sqrt{p}$ on the unit box) and ridge (10^{-16}–10^{-2} times N) are chosen over a wide grid by the leave-one-out error of proposition 1; POD+polynomial, degree up to six with at most 3000 features, degree and ridge chosen by leave-one-out; and a GP directly on the full field (Matérn-5/2 or squared-exponential, shared ARD kernel fitted on at most 300 runs), included only when $m \leq 2000$. A tuned neural network (`TunedMLP`, section 5) can be added as an optional member.

4.3. Cross-validated, tie-aware selection

Each member is scored by k-fold cross-validation ($k = 4$ by default) with all fitting, including hyper-parameter fitting, repeated inside each training fold. The member with the smallest mean cross-validated error (1) is selected, and all members within the tie band of section 3 are reported as tied. Members that fail to fit receive infinite error.

Algorithm 1: SurroSelect (default settings)

Input: runs $(\boldsymbol{\mu}_i, \mathbf{u}_i)_{i \le N}$, folds k, tie band $\tau = 0.1$ decade, optional target error ε

Scale inputs to the unit box; split the runs into k folds;

foreach *pool member* c **do**

  fit c on each training fold and score it on the held-out fold; $e_c \leftarrow$ mean error (1);

$c^\star \leftarrow \arg\min_c e_c$; tied set $\leftarrow \{c : \ e_c \le 10^\tau e_{c^\star}\}$; refit $c^\star$ on all N runs;

Basis floor $\phi \leftarrow$ mean projection error of held-out fields onto a rank-32 POD basis; flag the basis if $e_{c^\dagger} \le 1.25\,\phi$ for the best POD-based $c^\dagger$;

Learning curve: errors of $c^\star$ at $n \in \{N/8, N/4, N/2, N\}$ (training-fold sizes); fit $\log e = \log A - b \log n$ on the last three points (and on all four); forecast $e(4N)$; if ε is given, return $(A/\varepsilon)^{1/b}$ unless $b < 0.1$ or the result exceeds $16N$;

return $c^\star$*, tied set, basis flag, forecast*

4.4. Basis check

On each fold, a rank-32 POD basis is fitted on the training part and the held-out fields are projected onto it; the mean relative error of this projection is the *basis floor*. If the best POD-based member's cross-validated error is within 25% of the floor, the tool flags the basis as limiting and recommends a full-field model, a larger basis, or a nonlinear output map. The reason is proposition 2.

4.5. Budget forecast

For the selected member, cross-validated errors are computed at 1/8, 1/4, 1/2 and all of the training folds. A power law $e(n) = A\,n^{-b}$ is fitted by least squares in log–log coordinates to the last three points (central forecast) and to all four (alternative); the lower and upper of the two forecasts form a range. The number of runs needed for a target error ε is $(A/\varepsilon)^{1/b}$; the tool declines to answer if $b < 0.1$ or if the answer exceeds sixteen times the current number of runs. This is an extrapolation and is validated in **??**, not assumed.

4.6. Theory

Two results are standard and are included because the tool relies on them; proofs are in the Supplement (section S1supplement.pdf).

Proposition 1 (Closed-form leave-one-out)**.** *For a ridge smoother with hat matrix H (so $\hat{\mathbf{a}} = H\mathbf{a}$ on the training inputs), the leave-one-out prediction error at run i equals $(a_i - \hat{a}_i)/(1 - H_{ii})$. For $H = K(K + \lambda NI)^{-1}$ with $K = Q\,\mathrm{diag}(w)Q^\top$, $1 - H_{ii} = \sum_j Q_{ij}^2\, \lambda N/(w_j + \lambda N)$, which can be evaluated without cancellation when λN is tiny.*

Proposition 2 (Basis floor)**.** *Let $\hat{\mathbf{u}}$ be any model whose outputs lie in the affine subspace $\bar{\mathbf{u}}+\mathrm{span}(V_r)$. Then for every run, $\|\hat{\mathbf{u}}-\mathbf{u}\|_2 \geq \|(I-V_rV_r^\top)(\mathbf{u}-\bar{\mathbf{u}})\|_2$, so the mean relative error of $\hat{\mathbf{u}}$ is at least the mean relative projection error of the basis, whatever the regressor.*

The first is the PRESS identity for ridge regression [32]; the second is the optimality of orthogonal projection. The forecast has no guarantee: it is exact only if the error follows a power law, and **??** tests how often it is useful.

5. Experimental Framework

5.1. Setup

Data protocol.. For each problem we generate 4096 candidate training inputs, 1000 held-out test inputs and 500 extrapolation inputs (coordinates drawn from $[-0.25, 1.25]^p$, rejected if all lie inside the unit box) from streams seeded by the problem name; solutions are computed once and cached. A seed $s \in \{0, 1, 2\}$ fixes a permutation of the pool, and the training set for budget N is its first N elements, so training sets are nested within a seed. Budgets are $N \in \{32, \ldots, 1024\}$ for 24 problems and $\{64, 256, 1024\}$ for the others.

Problems.. Table 1 lists the 44 problems. 36 come from our own solvers, each verified against exact or manufactured solutions or by grid refinement; they are grouped into 6 classes named after application areas (the classes are our own groupings of standard PDE families; nothing was calibrated to a real application). 8 problems were added in the final stage and solved with the independent finite-element library `scikit-fem` (version 12.0.2); their unit tests compare with a series solution, a scaling law, load reversal, and mesh and time-step convergence. All data are simulated.

Hardware and software.. All runs used 4 CPU cores (Intel Xeon @ 2.10GHz), no GPU, Python 3.13.16, NumPy 2.5.3, SciPy 1.18.1, scikit-learn 1.9.1, PyTorch 2.14.1 and Optuna 5.0.0; each model fit used one thread.

Table 1: The 44 test problems by class. "Inputs" is the number of parameters p, "Outputs" the number of field values m.

Class	Problems	Inputs	Outputs	Solver
Composite heat conduction	2	4–9	256	scikit-fem (third party)
Convection–diffusion	2	3–4	256	scikit-fem (third party)
Elastic L-shaped plate	2	4–7	130	scikit-fem (third party)
Transient heat, moving source	2	3–5	192	scikit-fem (third party)
Flow through porous rock	9	3–32	256	our code
Heat conduction	9	3–8	128–256	our code
Pollutant or heat carried by a flow	4	4	128–256	our code
Spreading fronts and reactions	4	3–4	128–256	our code
Thermal runaway	2	3	256	our code
Waves, shocks, acoustics	8	3–4	128–256	our code

5.2. Metrics

The error is (1). The comparison between a group A (here the reduced-basis group) and a group B (the direct networks) is

$$\bar{y} = \frac{1}{S} \sum_{s=1}^{S} \log_{10} \frac{\mathrm{err}_B^{(s)}}{\mathrm{err}_A^{(s)}}, \tag{3}$$

over seeds, with the best member of each group chosen on validation data; $\bar{y} > 0.1$ means A is clearly better, $|\bar{y}| \le 0.1$ is a tie. For the tool, the *regret* of a cell is $\mathrm{err}(\text{selected}) / \min_{c \in \text{pool}} \mathrm{err}(c)$, with all errors measured on the 1000 test inputs; the *forecast ratio* of a case is $\hat{e}(4N_0)/e_{\text{actual}}(4N_0)$, where the actual error is that of the same pool member refit on $4N_0$ runs. Cell-wise values are geometric means over the three seeds.

5.3. Baselines

- *Five benchmark methods* under one tuning protocol: POD+GP, POD+polynomial, POD+MLP (reduced-basis group) and a direct MLP and a bottleneck MLP (network group). Each gets eight random-search trials (the first is a fixed default) scored on a random fifth of the training runs; networks use Adam for 1500 epochs in single precision.
- *Always POD+GP*: the benchmark's POD+GP, i.e. no selection.
- *Oracle*: the best pool member by test error (an upper bound; unavailable in practice).

Table 2: Pre-registered hypotheses for the tool and their pass criteria (cells: 8 problems $\times$ $N \in \{64, 256, 1024\} = 24$).

ID	Statement	Pass criterion
H1a	Cross-validated selection is close to the oracle	median regret ≤ 1.5 and $\geq 75\%$ of cells ≤ 2
H1b	Not worse than always using POD+GP	median ratio ≤ 1
H1c	At least as good as the benchmark's own selection	median ratio ≤ 1; more cells better than worse by $> 10\%$
H2	The forecast is usable	$4N_0$ forecast within a factor 2 in $\geq 70\%$ of 48 cases; reach-target classification $\geq 75\%$
H3	The basis flag identifies cases where a full-field model is better	$\geq 75\%$ of flagged cells (if at least five)
H4	A strongly tuned network rarely beats the tool	better by $> 25\%$ in at most 4 of 16 cells
H5	The benchmark finding holds on independent problems	at least 5 of 8 problems reduced-basis-clear at $N = 1024$

- *Strongly tuned network*: `TunedMLP`: Optuna TPE with successive-halving pruning [22, 21], 16 trials over output type (full field or POD rank 8/16/32), depth 2–4, width 64–256, activation, learning rate and weight decay; full-batch float64 training; final refit on all runs with four times the epochs and 200 L-BFGS iterations. It receives twice the tuning budget of any other method and is selected on a validation split of the training runs only.

Controls for the mechanism study are described in **??**.

5.4. Hypotheses and how each is tested

The hypotheses, metrics and pass/fail criteria below were committed to version control before the corresponding results (commit hashes in Supplement, section S2supplement.pdf); table 2 lists them. Earlier-stage hypotheses (pilot diagnostic, mechanism controls, longer training) are in the Supplement (section S3supplement.pdf). Binomial proportions are given with Wilson 95% intervals [33]; medians with percentile bootstrap intervals (2000 resamples over cells); the tests against the thresholds are one-sided exact binomial tests with Holm adjustment [34] over the three that admit one. STUB STUB

Acknowledgments

Use of AI tools. The study design, code, analyses, figures and the drafting of this paper were carried out with an AI system (Claude, Anthropic) acting as a research assistant under the authors' direction. The internal "reviews" and audits mentioned in the supplement were performed by AI agents, not by human reviewers. The authors are responsible for the content, have checked it (to be confirmed by the authors before submission), and all authors must approve the submission. [Funding, other acknowledgments and competing-interest statement: to be supplied by the authors.]

Data and code availability

All data are generated by the code in the project repository from problem names and seeds; raw outputs, aggregation scripts (which write every number in this paper) and figure scripts are included, and `code/run_all.py` reruns everything. The tool is the `surroselect` package (MIT licence; copyright holder to be set by the authors). A permanent archive with a DOI has not been created yet: [authors to provide the public repository link or archive DOI before submission].

References

[1] J. S. Hesthaven, S. Ubbiali, Non-intrusive reduced order modeling of nonlinear problems using neural networks, Journal of Computational Physics 363 (2018) 55–78. doi:10.1016/j.jcp.2018.02.037.

[2] K. Bhattacharya, B. Hosseini, N. B. Kovachki, A. M. Stuart, Model reduction and neural networks for parametric PDEs, SMAI Journal of Computational Mathematics 7 (2021) 121–157. doi:10.5802/smai-jcm.74.

[3] M. Guo, J. S. Hesthaven, Reduced order modeling for nonlinear structural analysis using Gaussian process regression, Computer Methods in Applied Mechanics and Engineering 341 (2018) 807–826. doi:10.1016/j.cma.2018.07.017.

[4] S. Fresca, A. Manzoni, POD-DL-ROM: enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition, Computer Methods in Applied Mechanics and Engineering 388 (2022) 114181. doi:10.1016/j.cma.2021.114181.

[5] N. R. Franco, A. Manzoni, P. Zunino, A deep learning approach to reduced order modelling of parameter dependent partial differential equations, Mathematics of Computation 92 (340) (2023) 483–524. doi:10.1090/mcom/3781.

[6] N. McGreivy, A. Hakim, Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations, Nature Machine Intelligence 6 (10) (2024) 1256–1269. doi:10.1038/s42256-024-00897-5.

[7] L. Lu, P. Jin, G. Pang, Z. Zhang, G. E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, Nature Machine Intelligence 3 (2021) 218–229. doi:10.1038/s42256-021-00302-5.

[8] Z. Li, N. Kovachki, K. Azizzadenesheli, et al., Fourier neural operator for parametric partial differential equations, in: International Conference on Learning Representations, 2021, arXiv:2010.08895.

[9] P. Batlle, M. Darcy, B. Hosseini, H. Owhadi, Kernel methods are competitive for operator learning, Journal of Computational Physics 496 (2024) 112549. doi:10.1016/j.jcp.2023.112549.

[10] J. Westermann, B. Huber, T. O'Leary-Roseberry, J. Zech, Performance of neural and polynomial operator surrogates, arXiv:2604.00689 (preprint) (2026).

[11] A. Bērziņš, J. Helmig, F. Key, S. Elgeti, Standardized non-intrusive reduced order modeling using different regression models with application to complex flow problems, arXiv:2006.13706 (preprint) (2020).

[12] C. Mora, A. Yousefpour, S. Hosseinmardi, H. Owhadi, R. Bostanabad, Operator learning with Gaussian processes, Computer Methods in Applied Mechanics and Engineering 434 (2025) 117581. doi:10.1016/j.cma.2024.117581.

[13] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar, Neural operator: learning maps between function spaces with applications to PDEs, Journal of Machine Learning Research 24 (89) (2023) 1–97.

[14] A. Cohen, R. DeVore, Approximation of high-dimensional parametric PDEs, Acta Numerica 24 (2015) 1–159. doi:10.1017/s0962492915000033.

[15] C. Greif, K. Urban, Decay of the Kolmogorov N-width for wave problems, Applied Mathematics Letters 96 (2019) 216–222. doi:10.1016/j.aml.2019.05.013.

[16] C. E. Rasmussen, C. K. I. Williams, Gaussian Processes for Machine Learning, MIT Press, 2006. doi:10.7551/mitpress/3206.001.0001.

[17] K. Duraisamy, Predictivity and utility of neural surrogates of multiscale PDEs, arXiv:2604.20061 (preprint) (2026).

[18] M. V. de Hoop, D. Z. Huang, E. Qian, A. M. Stuart, The cost-accuracy trade-off in operator learning with neural networks, Journal of Machine Learning 1 (3) (2022) 299–341. doi:10.4208/jml.220509.

[19] P. M. Wyder, J. Goldfeder, A. Yermakov, Y. Zhao, et al., Common task framework for a critical evaluation of scientific machine learning algorithms, arXiv:2510.23166 (preprint) (2025).

[20] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, Journal of Machine Learning Research 13 (2012) 281–305.

[21] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, A. Talwalkar, Hyperband: a novel bandit-based approach to hyperparameter optimization, Journal of Machine Learning Research 18 (2018) 1–52.

[22] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: a next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019. doi:10.1145/3292500.3330701.

[23] J. Dodge, S. Gururangan, D. Card, R. Schwartz, N. A. Smith, Show your work: improved reporting of experimental results, in: Proceedings of EMNLP-IJCNLP 2019, 2019, pp. 2185–2194. doi:10.18653/v1/d19-1224.

[24] X. Bouthillier, P. Delaunay, M. Bronzi, et al., Accounting for variance in machine learning benchmarks, in: Proceedings of Machine Learning and Systems (MLSys), 2021, arXiv:2103.03098.

[25] B. Williams, S. Cremaschi, Surrogate model selection for design space approximation and surrogate-based optimization, Computer Aided Chemical Engineering 47 (2019) 353–358. doi:10.1016/B978-0-12-818597-1.50056-4.

[26] F. Mohr, J. N. van Rijn, Learning curves for decision making in supervised machine learning: a survey, Machine Learning 113 (2024) 8371–8425. doi:10.1007/s10994-024-06619-7.

[27] J. S. Rosenfeld, A. Rosenfeld, Y. Belinkov, N. Shavit, A constructive prediction of the generalization error across scales, in: International Conference on Learning Representations, 2020, arXiv:1909.12673.

[28] R. Milk, S. Rave, F. Schindler, pyMOR – generic algorithms and interfaces for model order reduction, SIAM Journal on Scientific Computing 38 (5) (2016) S194–S216. doi:10.1137/15M1026614.

[29] S. Marelli, B. Sudret, UQLab: a framework for uncertainty quantification in Matlab, in: Vulnerability, Uncertainty, and Risk (ICVRAM 2014), 2014. doi:10.1061/9780784413609.257.

[30] T. Gustafsson, G. D. McBain, scikit-fem: a Python package for finite element assembly, Journal of Open Source Software 5 (52) (2020) 2369. doi:10.21105/joss.02369.

[31] M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, M. Niepert, PDEBench: an extensive benchmark for scientific machine learning, neurIPS 2022 Datasets and Benchmarks Track; arXiv:2210.07182 (2022).

[32] G. H. Golub, M. Heath, G. Wahba, Generalized cross-validation as a method for choosing a good ridge parameter, Technometrics 21 (2) (1979) 215–223. doi:10.1080/00401706.1979.10489751.

[33] E. B. Wilson, Probable inference, the law of succession, and statistical inference, Journal of the American Statistical Association 22 (158) (1927) 209–212. doi:10.1080/01621459.1927.10502953.

[34] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.